AI Smart Route Planner & Campus Navigator

Project Report

1. Introduction

The **AI Smart Route Planner & Campus Navigator** is a full-stack web application developed using **Node.js**, **Express.js**, **HTML**, **CSS**, and **JavaScript**. The project helps users find routes between important locations in the Tricity region, including Chandigarh, Mohali, Kharar, and Chandigarh University.

The project represents locations as a graph, where each location is a node and each road connection is an edge with a distance value. Graph algorithms such as **BFS**, **DFS**, and **Dijkstra's Algorithm** are used to explore routes and calculate the shortest path.

The system also includes a simple rule-based AI feature that explains why a route is suitable and simulates traffic conditions using basic logic.

2. Project Objectives

The main objectives of this project are:

- To represent Tricity locations using a graph data structure.
- To store connected locations with distances.
- To implement BFS traversal.
- To implement DFS traversal.
- To find the shortest route between a selected source and destination.
- To display route path, total distance, and number of visited nodes.
- To provide an intelligent route suggestion using simple rule-based AI.
- To compare BFS and DFS traversal behavior.
- To build a clean and responsive user interface.
- To create a beginner-friendly full-stack project structure.

3. Technologies Used

Technology	Purpose
Node.js	Runtime environment for backend JavaScript
Express.js	Backend server and API routing
HTML	Frontend page structure
CSS	Styling, layout, responsiveness, and animations
JavaScript	Frontend interactivity and backend graph logic
JSON	API request and response format

4. Project Structure

```
Project2/
  package.json
  package-lock.json
  server.js
  PROJECT_REPORT.md
  public/
    index.html
    styles.css
    script.js
```

File Description

File	Description
`server.js`	Contains Express server, graph data, BFS, DFS, Dijkstra, AI logic, and API routes
`package.json`	Stores project metadata, scripts, and dependencies
`package-lock.json`	Stores exact installed dependency versions
`public/index.html`	Contains frontend layout and UI elements
`public/styles.css`	Contains modern responsive design and animations
`public/script.js`	Handles frontend API calls and displays results
`PROJECT_REPORT.md`	Project documentation and report

---

5. Graph System

The project uses an **adjacency list** to represent the graph. In an adjacency list, each location stores a list of directly connected neighboring locations along with the distance between them.

Locations Included

Chandigarh

- Sector17
- SukhnaLake
- PGI
- Elante
- RockGarden

Mohali

- Phase7
- Phase5
- QuarkCity
- ISBT
- PCAStadium

- VRPunjab

Other Locations

- CU
- Kharar

Example Graph Representation

```
CU: [
  { node: "Kharar", distance: 7 },
  { node: "Phase7", distance: 16 }
]
```

This means:

- CU is connected to Kharar with a distance of 7 km.
- CU is connected to Phase7 with a distance of 16 km.

The complete graph is stored in `server.js`.

---

6. Algorithms Used

6.1 Breadth First Search

Breadth First Search, or **BFS**, explores a graph level by level. It first visits the source node, then all directly connected neighbors, then the neighbors of those neighbors.

Use in Project

In this project, BFS is used to:

- Traverse all connected locations from a selected source.
- Find a route with fewer stops for comparison.
- Show how graph exploration works level by level.

BFS Characteristics

Feature	Description
Data structure used	Queue
Traversal style	Level by level
Best for	Fewer-stop route discovery in unweighted graphs
Time complexity	O(V + E)

Here, `V` is the number of vertices or nodes, and `E` is the number of edges.

6.2 Depth First Search

Depth First Search, or **DFS**, explores a graph by going as deep as possible on one path before backtracking.

Use in Project

In this project, DFS is used to:

- Traverse all locations from a selected source.
- Compare traversal behavior with BFS.
- Show deep exploration of graph connections.

DFS Characteristics

Feature	Description
Data structure used	Recursion / Stack
Traversal style	Deep path first
Best for	Graph exploration and connectivity checking
Time complexity	O(V + E)

6.3 Dijkstra's Algorithm

Dijkstra's Algorithm is used to find the shortest path in a weighted graph. Since this project stores distances between locations, Dijkstra is the most suitable algorithm for finding the route with the minimum total distance.

Use in Project

In this project, Dijkstra's Algorithm is used in the main **Find Route** feature.

It returns:

- Shortest path
- Total distance
- Number of nodes visited
- AI route explanation
- BFS comparison route

Dijkstra Characteristics

Feature	Description
Graph type	Weighted graph
Main purpose	Shortest distance route
Data tracked	Distance and previous node
Output	Minimum distance path

---

7. Backend API Routes

The backend provides the following API routes:

7.1 `/locations`

Method

```
GET
```

Purpose

Returns all available locations for the frontend dropdown menus.

Example Response

```
{
  "locations": ["CU", "Elante", "ISBT", "Kharar", "PCAStadium"]
}
```

7.2 `/find-route`

Method

```
POST
```

Purpose

Finds the shortest route between source and destination using Dijkstra's Algorithm.

Example Request

```
{
  "source": "CU",
  "destination": "Sector17"
}
```

Example Response

```
{
  "algorithm": "Dijkstra",
  "path": ["CU", "Phase7", "PCAStadium", "Sector17"],
  "distance": 28,
  "visitedCount": 9,
  "traffic": "Moderate",
  "aiExplanation": "This route is better because it covers 28 km with 3 stops."
}
```

7.3 `/bfs`

Method

```
POST
```

Purpose

Performs BFS traversal from the selected source location.

Example Request

```
{
  "source": "CU",
  "destination": "Sector17"
}
```

Output

- BFS traversal order
- Number of visited nodes
- BFS route if destination is selected

7.4 `/dfs`

Method

```
POST
```

Purpose

Performs DFS traversal from the selected source location.

Example Request

```
{
  "source": "CU"
}
```

Output

- DFS traversal order
- Number of visited nodes

---

8. AI Feature

The AI feature in this project is rule-based and beginner-friendly. It does not use a complex machine learning model. Instead, it analyzes the selected route using simple conditions.

AI Logic Includes

- Total route distance
- Number of stops
- Comparison with BFS route
- Traffic simulation based on busy places

Busy Locations Used for Traffic Simulation

- Sector17
- Elante
- ISBT
- PCAStadium

Traffic Rules

Condition	Traffic Level
Route includes 3 or more busy places	High
Route includes 2 busy places	Moderate
Route includes 0 or 1 busy place	Light

Example AI Explanation

```
This route is better because it covers 28 km with 3 stops.
It also matches the simple BFS route distance for this trip.
Traffic looks moderate because the route touches a few busy locations.
```

---

9. Frontend Features

The frontend is built using HTML, CSS, and JavaScript.

Main UI Features

- Source dropdown
- Destination dropdown
- Find Route button
- BFS button
- DFS button
- Shortest path card
- Distance card
- Nodes visited card
- AI suggestion card
- BFS vs DFS comparison card
- Responsive layout for mobile and desktop

- Hover effects and simple animations
- Highlighted shortest path display

10. Sample Output

Input

```
Source: CU
Destination: Sector17
```

Output

```
Path: CU -> Phase7 -> PCAStadium -> Sector17
Total Distance: 28 km
Nodes Visited: 9
Traffic: Moderate
```

AI Explanation

```
This route is better because it covers 28 km with 3 stops.
It also matches the simple BFS route distance for this trip.
Traffic looks moderate because the route touches a few busy locations.
```

11. BFS vs DFS Comparison

Basis	BFS	DFS
Full form	Breadth First Search	Depth First Search
Approach	Visits nearby nodes first	Goes deep first
Data structure	Queue	Stack or recursion
Shortest path use	Useful in unweighted graphs	Not used for shortest path
Project use	Traversal and fewer-stop route comparison	Traversal comparison

In this project, BFS and DFS help students understand different graph traversal methods. Dijkstra is used for the final shortest-distance route because the graph has weighted edges.

12. How to Run the Project

Step 1: Open Project Folder

Open the following folder in VS Code:

```
P:\MCA\Semester 2\Projects\Project2
```

Step 2: Install Dependencies

Run this command in the VS Code terminal:

```
npm install
```

Step 3: Start the Server

Run:

```
npm start
```

Step 4: Open in Browser

Open this URL:

```
http://localhost:3000
```

---

13. Testing

The project was tested using API requests and browser interaction.

Test Case 1

Field	Value
Source	CU
Destination	Sector17
Expected result	Shortest route should be displayed
Actual result	CU -> Phase7 -> PCAStadium -> Sector17
Status	Passed

Test Case 2

Field	Value
Action	Click BFS
Expected result	BFS traversal order should be displayed
Actual result	BFS traversal displayed successfully
Status	Passed

Test Case 3

Field	Value
Action	Click DFS
Expected result	DFS traversal order should be displayed
Actual result	DFS traversal displayed successfully
Status	Passed

14. Advantages

- Simple and beginner-friendly implementation.
- Demonstrates practical use of graph algorithms.
- Includes full-stack development concepts.
- Provides a modern and responsive UI.
- Uses weighted graph routing.
- Includes simple AI-based explanation.
- Helps compare BFS, DFS, and Dijkstra.

15. Limitations

- Traffic conditions are simulated using simple rules.
- Distances are sample values, not live map distances.
- The project does not use real GPS or map APIs.
- Dijkstra implementation is simple and does not use a priority queue.
- The AI feature is rule-based, not machine-learning based.

16. Future Scope

The project can be improved in the future by adding:

- Real map integration using Google Maps or OpenStreetMap.
- Live traffic data.
- User login and saved routes.
- Admin panel to add or update locations.
- Priority queue optimization for Dijkstra's Algorithm.
- Voice-based navigation instructions.
- Estimated travel time calculation.
- Route filtering based on traffic, distance, or number of stops.

17. Conclusion

The **AI Smart Route Planner & Campus Navigator** successfully demonstrates how graph data structures and graph algorithms can be used in real-world navigation systems. The project allows users to select a source and destination, calculate the shortest route, view total distance, check visited nodes, and receive an AI-style route explanation.

The project is useful for understanding BFS, DFS, Dijkstra's Algorithm, full-stack development, API communication, and responsive UI design. It is simple enough for beginners while still covering important computer science and web development concepts.